

The background of the slide features a large, faint watermark of the Universidad Mayor de San Andrés (UMSS) logo. It is an inverted triangle with a white top section containing the text 'UMSS' in blue. The bottom two sections are red and contain a white geometric emblem of a star or flower. The entire watermark is semi-transparent.

UMSS

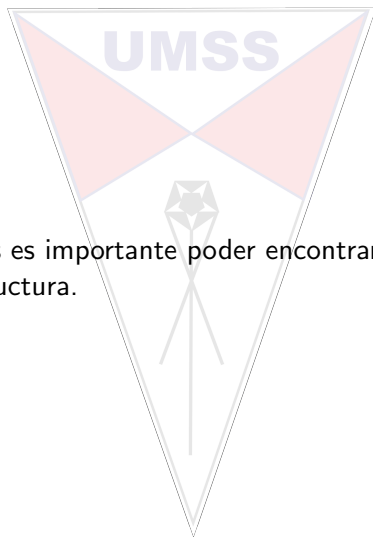
Clasificación y Búsquedas

Leticia Blanco

Departamento de Informática - Sistemas
UMSS

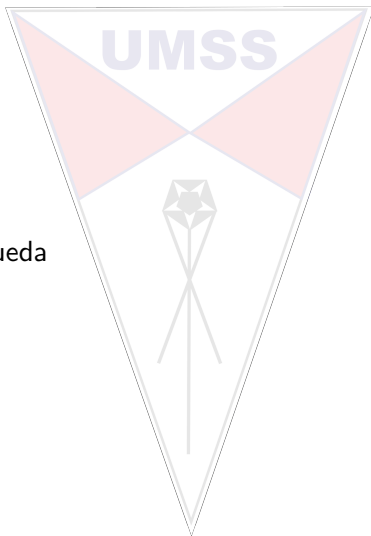
19 de septiembre de 2024

Motivación

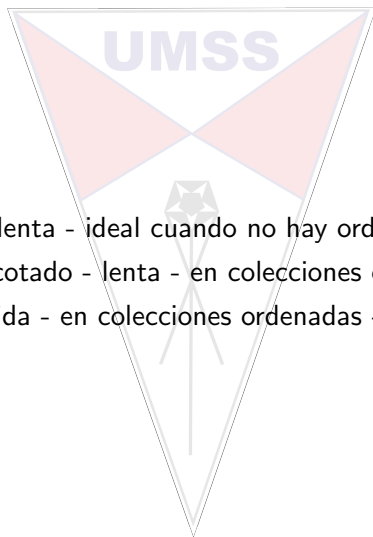


En las colecciones es importante poder encontrar datos, utilizarlos o cambiar su estructura.

Técnicas de búsqueda



Búsquedas



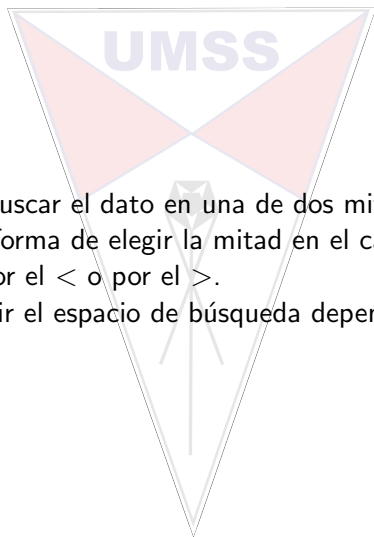
- ▶ Secuencial - lenta - ideal cuando no hay orden - $O(n)$
- ▶ Secuencial acotado - lenta - en colecciones ordenadas - $O(n)$
- ▶ Binaria - rápida - en colecciones ordenadas - $O(\log n)$

Búsqueda secuencial

La cantidad de comparaciones es igual a la cantidad de elementos que tiene la colección.

- ▶ Es exhaustiva y usualmente es útil cuando el filtro se aplica mas de una vez en el espacio de búsqueda.
- ▶ Esta condicionada por el cumplimiento del filtro y usualmente la búsqueda se corta una vez que se cumple la condición buscada.

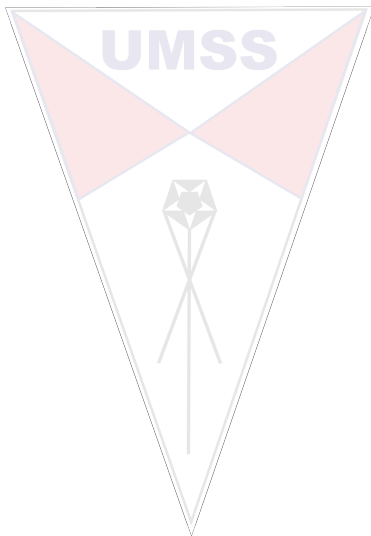
Búsqueda binaria



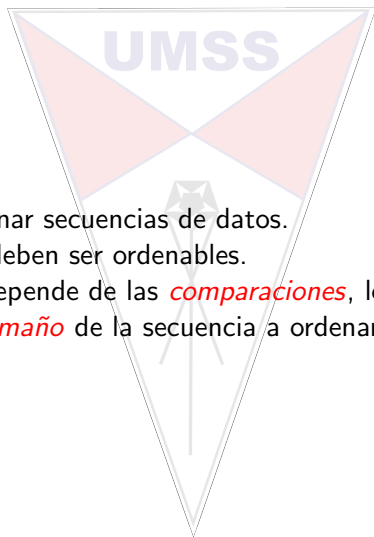
La estrategia es buscar el dato en una de dos mitades del espacio de búsqueda. La forma de elegir la mitad en el caso tradicional es la comparación por el $<$ o por el $>$.

La forma de dividir el espacio de búsqueda dependerá de lo que se quiere buscar.

Ordenamientos



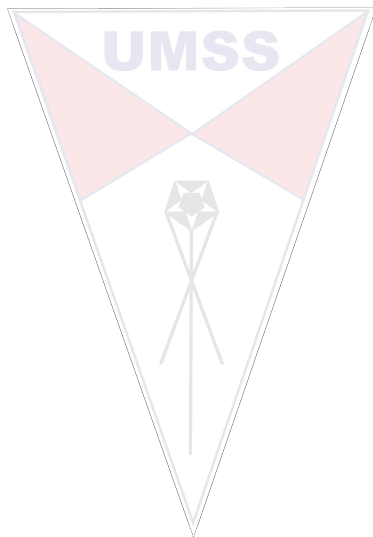
Ordenamiento



La misión es ordenar secuencias de datos.
Condición datos deben ser ordenables.
Cómo se mide? depende de las *comparaciones*, los movimientos *intercambios* y *tamaño* de la secuencia a ordenar.

Métodos lentos

- ▶ Burbuja
- ▶ Selección
- ▶ Inserción



Burbuja

El más inocente método

- ▶ Ubicar en la primera posición el elemento más pequeño de los n elementos, comparándolo con los restantes ($n - 1$).
- ▶ Ubicar en la segunda posición el elemento más pequeño de los restantes $n - 1$ elementos, comparándolo con los restantes ($n - 2$).

Y en general:

- ▶ Ubicar en la i - *sima* posición el elemento más pequeño de los restantes $n - i$ elementos, comparándolo con los restantes ($n - i - 1$).

Complejidad $O(n^2)$

Selección

Mejora tiene menos intercambios.....

- ▶ Buscar el mínimo elemento de la lista
- ▶ Intercambiarlo con el primero
- ▶ Buscar el mínimo en el resto de la lista
- ▶ Intercambiarlo con el segundo

Y en general:

- ▶ Buscar el mínimo elemento entre una posición i y el final de la lista
- ▶ Intercambiar el mínimo con el elemento de la posición i

Complejidad $O(n^2)$

Estrategia de Burbuja, Selección..

Lo que se rescata es la forma de conseguir el objetivos, que es por comparación de todos contra todos, no necesariamente para ordenar, sino para otro tipo de ejercicios.

Lo bueno de la estrategia es que es exhaustiva.

Mejoras

Ordenamiento indirecto: posiciones se evita intercambios

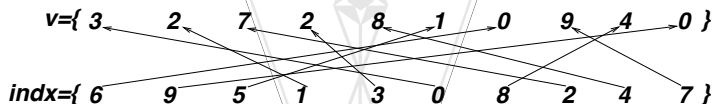


Figura 5.1: Ordenamiento indirecto.

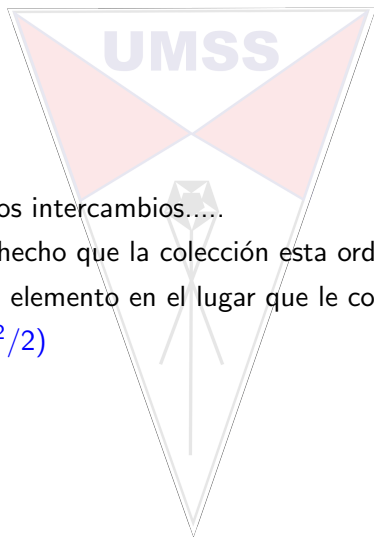
Complejidad $O(n^2)$

Inserción

Mejora tiene menos intercambios.....

- ▶ Se parte del hecho que la colección esta ordenada
- ▶ Se inserta un elemento en el lugar que le corresponde

Complejidad $O(n^2/2)$



Métodos rápidos

quicksort

- ▶ seleccionar un pivote
- ▶ menores a la izquierda de pivote
- ▶ mayores a la derecha de pivote

En una vuelta el pivote quedaria ordenado respecto a los otros elementos.

- ▶ repetir el proceso con los elementos que son menores a pivote
- ▶ repetir el proceso con los elementos que son mayores a pivote

Métodos rápidos

Rápidos: *quicksort*

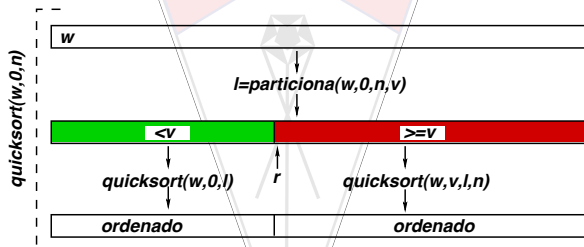


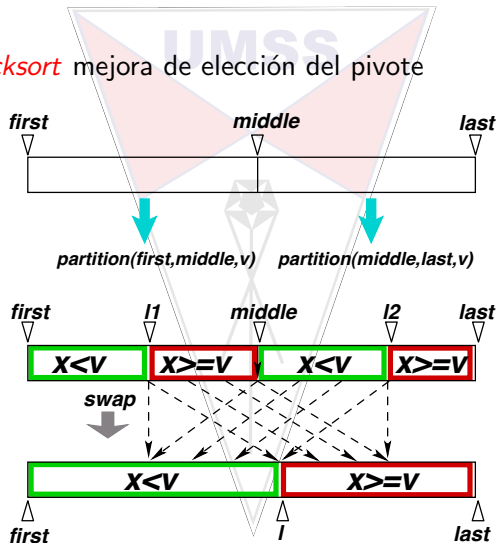
Figura 5.2: Esquema “dividir para vencer” para el algoritmo de ordenamiento rápido, *quick-sort*.

Métodos



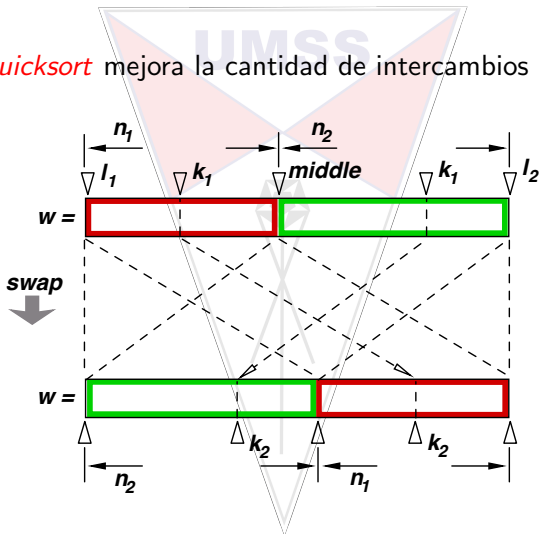
Métodos

Ejemplo *quicksort* mejora de elección del pivote



Métodos

Ejemplo *quicksort* mejora la cantidad de intercambios

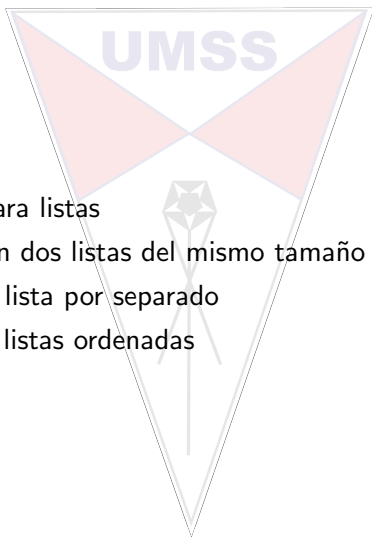


Complejidad: $O(n \log n) \Rightarrow O((n \log n)^2)$

Métodos

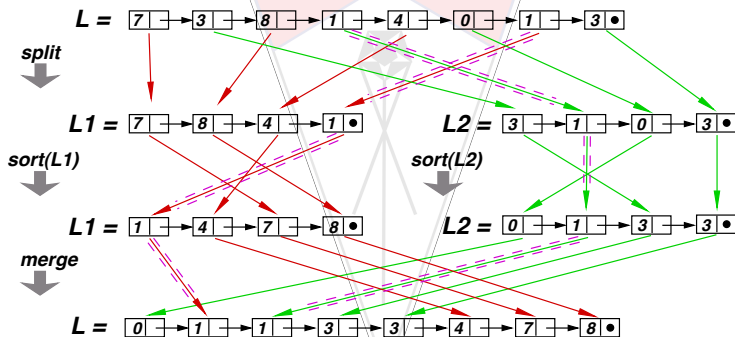
mergesort ideal para listas

- ▶ dividir lista en dos listas del mismo tamaño
- ▶ ordenar cada lista por separado
- ▶ intercalar las listas ordenadas



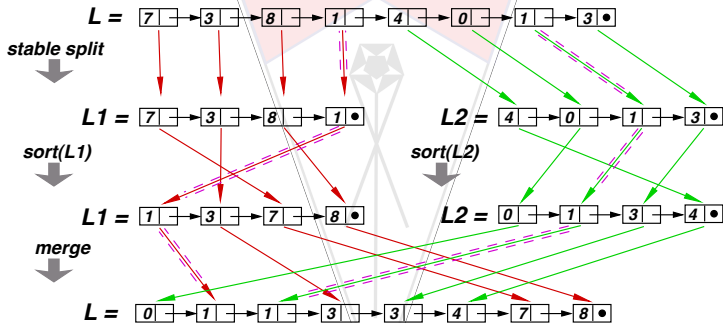
Métodos

Ejemplo *mergesort*



Métodos

Ejemplo *mergesort*



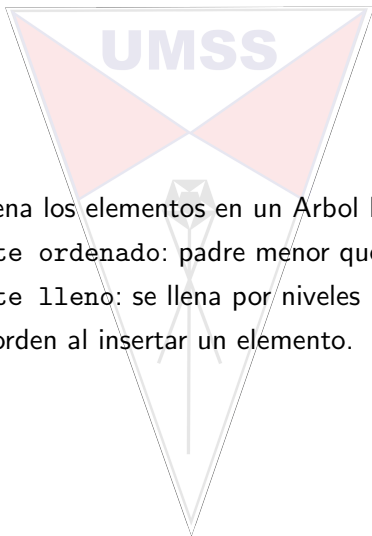
Complejidad: $O(n \log n)$

Estrategia de Quick y Merge

La estrategia en estos dos métodos es la forma de afrontar el problema, y es que se divide en mitades las opciones y procesar estas mitades.

Hay que recordar que no necesariamente el objetivo es ordenar.

Métodos



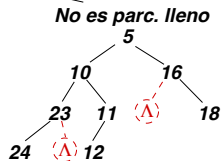
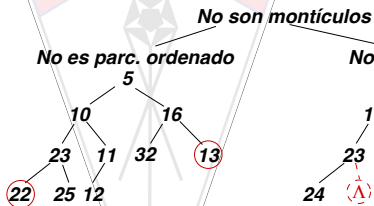
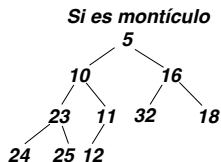
Montículo Almacena los elementos en un Arbol Binario en el que:

- ▶ parcialmente ordenado: padre menor que sus hijos
- ▶ parcialmente lleno: se llena por niveles

Se va generando orden al insertar un elemento.

Métodos

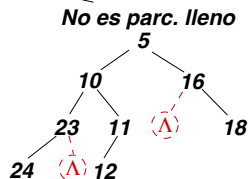
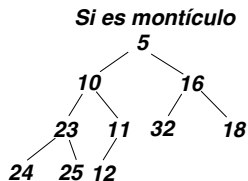
Montículo Diferencia entre arboles binarios



Problema? al insertar

Métodos

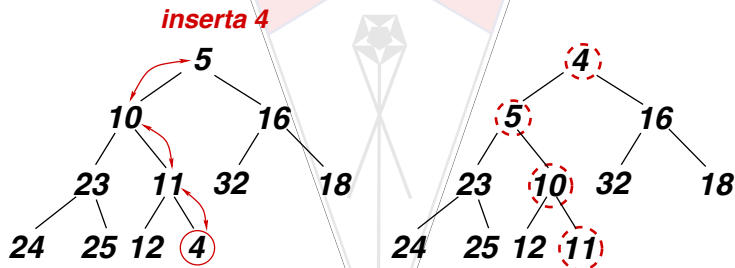
Montículo Diferencia entre arboles binarios



Problema? al insertar

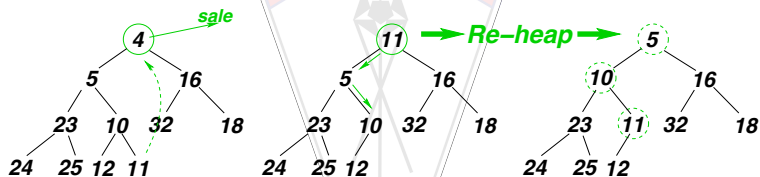
Métodos

Montículo: inserción



Métodos

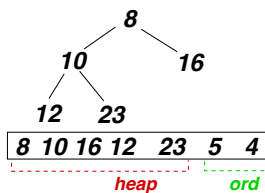
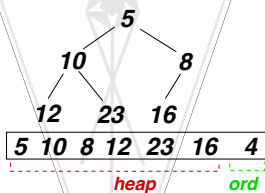
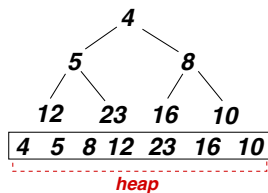
Montículo: eliminación



Ventaja el arbol es equilibrado

Métodos

Montículo: orden?

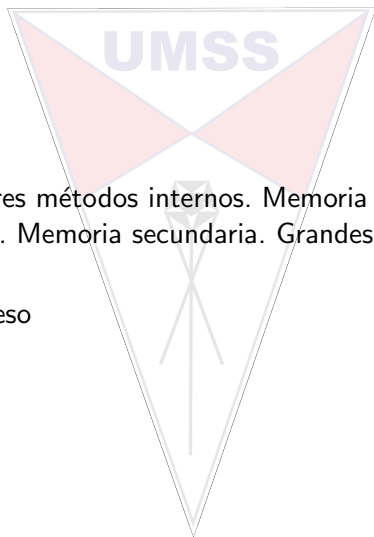


Complejidad: $O(n \log n)$

Estrategia de Montículo

Aprovechar las propiedades de un árbol binario y sus accesos, cuando cumple con algunas condiciones. La ventaja es que se puede linealizar sin pérdida de información y además con accesos directos que garantizan un $O(\log n)$.

Métodos



Todos los anteriores métodos internos. Memoria primaria
Métodos externos. Memoria secundaria. Grandes volúmenes de datos.

- costo de acceso

Mejor mergesort

Métodos

Averiguar otros métodos externos.



Counting sort

Ideal cuando se sabe que los elementos están en un rango predeterminado $[0.k]$, y por supuesto este rango es razonablemente pequeño.

Para este proceso se usarán tres arreglos:

- ▶ **A**: que es la entrada de datos desordenada
- ▶ **B**: que es la salida con los datos ordenados
- ▶ **C**: que es el arreglo de conteo de ocurrencias

Counting sort



	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0

Counting sort



	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	0	1	1	0	0	0	1	1	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	1	1	0	0	0	1	1	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	0	0	0	1	1	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	0	0	1	1	1	0	0	0	0	0	0	0

Counting sort



	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	0	0	1	2	1	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	0	0	1	2	2	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	0	0	1	2	2	0	1	0	0	0	0	0

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	0	0	1	2	2	0	1	1	0	0	0	0

Counting sort



	1										2													
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5									
C	0	1	2	1	0	0	1	2	2	0	2	1	0	0	0	0								

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2					
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5									
C	0	1	2	1	0	0	1	2	2	0	2	1	1	0	0	0								

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2					
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5									
C	0	1	2	1	0	0	1	2	3	0	2	1	1	0	0	0								

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2					
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5									
C	0	1	2	1	0	0	1	2	3	1	2	1	1	0	0	0								

Counting sort



	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5
C	0	1	2	1	0	0	1	2	3	1	3	1	1	0	0

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5
C	0	1	2	1	0	0	1	2	3	1	3	2	1	0	0

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5
C	0	1	2	1	0	0	1	2	3	1	3	2	1	0	1

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	1	2	3	4	5
C	0	1	2	1	0	0	1	2	3	1	3	2	1	0	1

Counting sort



	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	1	0	1	2	3	1	3	2	1	0	1	1

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	2	1	1	1	1	2	3	1	3	2	1	0	1	1

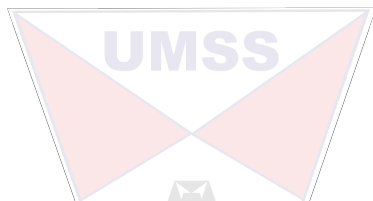
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	3	1	1	1	1	2	3	1	3	2	1	0	1	1

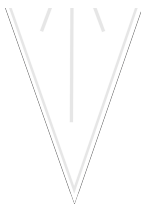
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	3	2	1	1	1	2	3	1	3	2	1	0	1	1

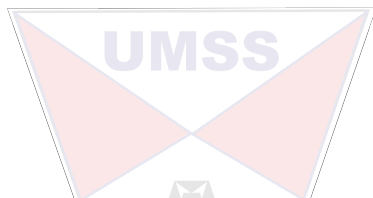
Counting sort



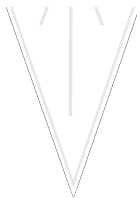
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
C	0	1	3	2	1	1	1	3	3	1	3	2	1	0	1	1								



Counting sort



											1					
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
C	0	1	3	2	1	1	1	3	3	1	3	2	1	0	1	1
	0	1	4	6	7	8	9	12	15	16	19	21	22	22	23	24



Counting sort



C 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
0 1 4 6 7 8 9 12 15 16 19 21 22 22 23 24

B 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
0 0

A 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
3 7 8 2 1 2 6 7 8 10 11 10 12 8 9 10 11 14 15 4 5 2 3 7

C 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
0 1 4 6 7 8 9 12 15 16 19 21 22 22 23 24
 11

B 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
0 0 0 0 0 0 0 0 0 0 0 7 0 0 0 0 0 0 0 0 0 0 0 0

A 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
3 7 8 2 1 2 6 7 8 10 11 10 12 8 9 10 11 14 15 4 5 2 3 7

C 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
0 1 4 6 7 8 9 11 15 16 19 21 22 22 23 24
 5

B 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
0 0 0 0 0 3 0 0 0 0 0 7 0 0 0 0 0 0 0 0 0 0 0 0

Counting sort



	1										2													
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

	0	1	2	3	4	5	6	7	8	9	10	1	2	3	4	5
C	0	1	4	5	7	8	9	11	15	16	19	21	22	22	23	24
			3													

	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2	1	2	3	4
B	0	0	0	2	0	3	0	0	0	0	7	0	0	0	0	0	0	0	0	0	0	0	0

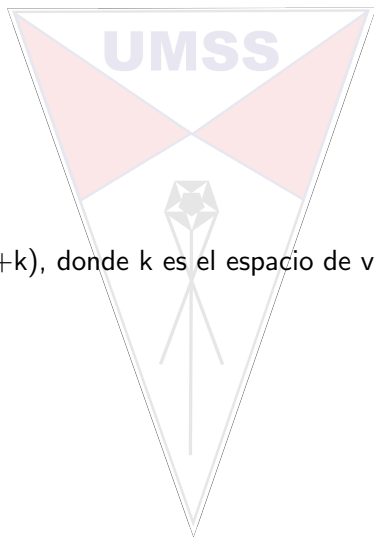
	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9	2	1	2	3	4	
A	3	7	8	2	1	2	6	7	8	10	11	10	12	8	9	10	11	14	15	4	5	2	3	7

		0	1	2	3	4	5	6	7	8	9	10	1	2	3	4	5
C	0	0	1	3	5	7	8	9	11	15	16	19	21	22	22	23	24
			0	2	4	6	7	8	10	14	15	18	20	21		22	23
				1					9	13		17	19				
									12			16					

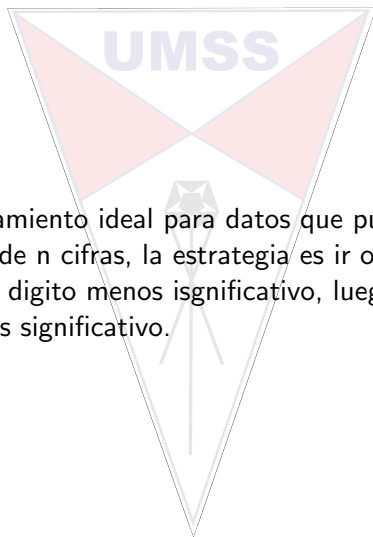
									1										2					
B	1	2	2	2	3	3	4	5	6	7	7	7	8	8	8	9	10	10	10	11	11	12	14	15

Counting sort

Complejidad $O(n+k)$, donde k es el espacio de valores



Radix sort



Metodo de ordenamiento ideal para datos que pueden expresarse como un numero de n cifras, la estrategia es ir ordenando empezando por el digito menos isgnificativo, luego el proximo hasta llegar al mas significativo.

Radix sort



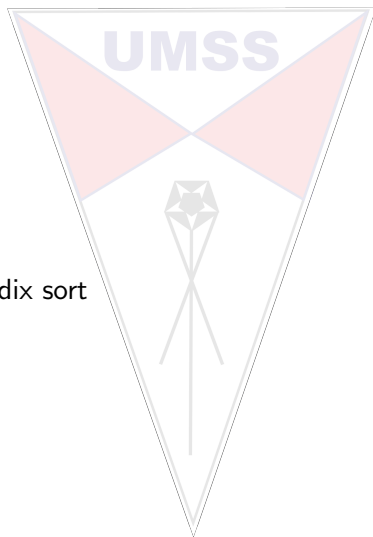
329		720		720		329
457		355		329		355
657		436		436		436
839	>	457	>	839	>	457
436		657		355		657
720		329		457		720
355		839		657		839

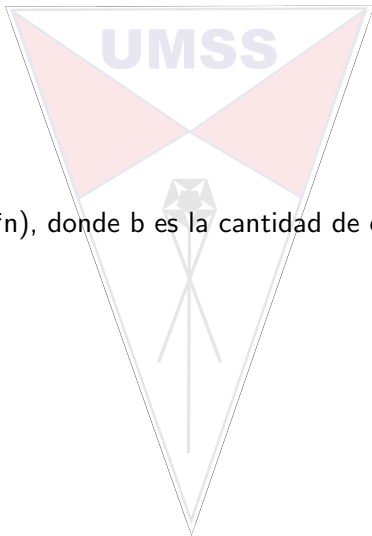


Radix sort

Tarea

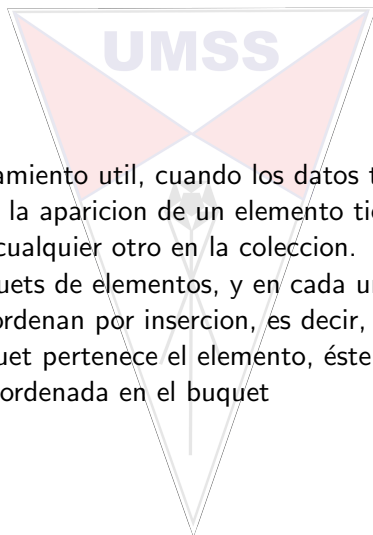
Implementar el radix sort





Complejidad $O(b \cdot n)$, donde b es la cantidad de cifras de la clave

Bouquet sort



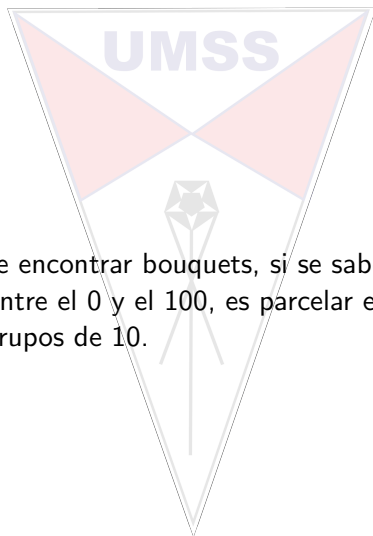
Metodo de ordenamiento util, cuando los datos tiene distribucion uniforme. Cuando la aparicion de un elemento tiene la misma probabilidad que cualquier otro en la coleccion.

Se consigue bouquets de elementos, y en cada uno de los bouquets los elementos se ordenan por insercion, es decir, una vez que se ubica a qué bouquet pertenece el elemento, éste elemeneot se lo insertar de forma ordenada en el buquet

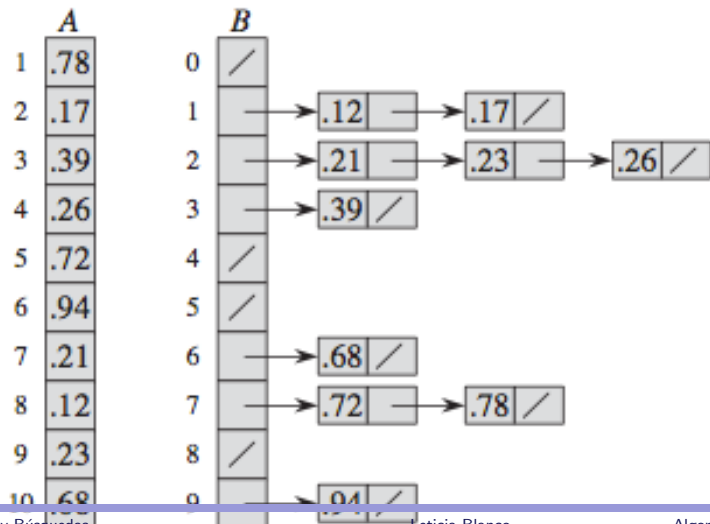
Bouquet sort

Example

Una forma fácil de encontrar bouquets, si se sabe que los elementos estan entre el 0 y el 100, es parcelar el total de posibilidades en grupos de 10.



Bouquet sort



Bouquet sort

Complejidad $O(n)$

